Sprint 1

Marchini Claudio, Tomasi Cesare

Introduction

This sprint builds on top of the work of sprint 0 and aims to build the core buisness of the system, in particular:

- The main *cargoservice* actor
- The *sonar* subsystem
- The *IOPort* and *pushbutton* interfaces

Requirements

Requirement Analysis

The requirement analysis has already been specified in the sprint 0, this particular sprint does not require anything else to be specified in this section.

Problem Analysis

The **cargoservice** actor works as the main component of the system, the requirement analysis specifies the workflow of the component, here we will specify it again in order to focus on many details with more precision:

1. It listens for a **request to load** received by the *pushbutton*
2. It checks the state of the **IOPort**, and decides which answer to send back, as

defined in the sprint 0

3. If a slot is free, it waits for the container to be placed in the *sensor area* and then delegates the *cargorobot* service to move the container to **slot-5**
4. It then moves the container from **slot-5** to the reserved slot again with the use of the **cargorobot**

```
QActor cargoservice context ctxcargoservice {
    [# val TimeoutMillis = 30000 #]
    [# val ReservedSlot = 0 #]

    State s0 disengaged {
        // Wait for a request to load

        if [# ioport occupied #] {
            //send retrylater
        } else if [# slots occupied #] {
            //send rejected
        } else {
            //reserve slot
            //set reserved slot var
            //send accepted(slot id)
            Goto engaged
        }

    }

    State engaged {
        //call blink led service
    }
    Transition t0
        whenTime TimeoutMillis -> disengaged
        whenMessage IOPortDeposited -> moveRobot

    State moveRobot {
        // call robot service from IOPort to slot 5
        // wait 3s to mark the container
        // call robot service from slot 5 to slot ReservedSlot

        //stop blink led service
        Goto disengaged
    }
}
```

The **sonar** is a HC-SR04 and is connected to a *Raspberry Pi Pico W*, by nature the hardware has a few characteristics:

- The raspberry is too lightweight to support a full JVM needed to run *qak* directly, so we have to use either cpp or micropython as the language
- Since *qak* is not a possibility the comunication must be implemented with a protocol, preferably one that *qak* suports out of the box
- *qak* supports a plethora of protocols, TCP, CoAP, MQTT, ecc..

The software house has decided to use **MQTT** as the protocol of choice because:

- It uses a centralized broker/client system, allowing the system to easily change or add devices in the future (multiple IOPorts or a device substituion), otherwise at least one of the nodes should know how to reach the other one, which is not always granted
- The broker is not particulary resource intensive and can easily be deployed along the rest of the service, likely in the same node as the main *cargoservice* actor
- The topic system keeps messages from various nodes neatly separated

```
def connectWifi():
    #connect to wifi

    print("Connected:", wlan.ifconfig())

def connectMqtt():
    # create Mqtt client
    # connect to Mqtt broker
    return client

def measureDistance():
    # measure distance using sonar
    return distance

TRIG = Pin(3, Pin.OUT)
ECHO = Pin(2, Pin.IN)

connectWifi()
client = connectMqtt()

while True:
    try:
        d = measureDistance()
        if d is not None:
            # create formatted msg
            client.publish(TOPIC, msg.encode() )

        time.sleep(1)
    except Exception as e:
        print("Exception: ", type(e).__name__, e)
```

- We have assumed that the Board will be connected through Wifi
- The configurations parameters (Wifi SSID, Password, MQTT broker, topic, ect..) will be set in a *.env* file in order to enhance reusability.
- The *msg* will be formatted as specified by the documentation of unibo.basicomm23-1.0, a library developed by our software house, already used by *qak*

Once we have the raw distance data we have two options:

- Send to the rest of the system the distance, it will then be a responsability of each

component to interpret the data accordingly as defined in the requirements.

- Create an intermediary that transforms the raw data into messages that respect the semantics of the requirements that will be sent to the rest of the system.

We decided to create an intermediary wrapper in order to have a single point where the distance is parsed, this allows to avoid repeating code, making it more robust and easily adjustable in the future.

```
Event serviceWorking : serviceWorking(X)
Event outOfService : outOfService(X)

Dispatch containerInIOPort : containerInIoPort(X)

QActor sonarwrapper context ctxcargoservice {
    State s0 initial {
        // listen to the sonar messages and send the right events/dispatch
    }
}
```

We choose to make the display messages events to permit a future expansion through extensibility. The containerInIOPort, instead is a dispatch because the only receiver should be the *cargoservice* actor.

Since the **IOPort** has to mantain the current status of the system at any moment the web page will need a **WebSocket** connection to ensure it receives updates, also receive the state of the *hold* our implementation in the requirement analysis will also need a listener that will forward any change via WebSocket to all connected clients.

```
const socket = new WebSocket("ws://localhost:8080");

const requestSpan = document.getElementById("request-result");
const statusSpan = document.getElementById("current-status");

socket.addEventListener("open", (event) => {
  //Request initial state
});

socket.addEventListener("message", (event) => {
  // Dispatch by message type and update right span
});
```

Test Plans

Project

Testing

Deployment

Maintenance

Marchini Claudio
claudio.marchini@studio.unibo.it

Tomasi Cesare
cesare.tomasi@studio.unibo.it

SignedSnow0/CargoService